Relatório de aprendizagem — Aula 07

Expressões, atribuição e estruturas de controle
VITOR HUGO ASSIS DA SILVA · RA 241400232 · Turma ESOFT6B
04/11/2026

1. O tema da aula

Esta aula tratou de regras que parecem pequenas quando vistas isoladamente, mas que decidem o resultado de um programa: a ordem das operações, as conversões entre tipos, a avaliação de condições e o caminho escolhido pelas estruturas de controle. O ponto principal não é decorar uma lista de operadores. É conseguir olhar para um trecho de código, prever o que ele faz e justificar a previsão pela regra da linguagem.

A comparação entre linguagens deixou claro que a aparência do código pode enganar. A expressão 3 > 2 > 1, por exemplo, funciona como comparação encadeada em Python, mas em C a primeira comparação produz 1 e a segunda testa 1 > 1. O mesmo texto gera respostas diferentes. Também há situações em que o resultado parece certo em um teste e falha em outro, principalmente quando há conversão implícita ou efeito colateral.

Consequências fora dos exercícios

Os materiais da aula citaram três casos para ligar essas regras ao mundo real. No episódio conhecido como “goto fail”, uma linha de desvio duplicada fez uma verificação de assinatura ser ignorada. Nos tocadores Zune, uma combinação específica de data manteve um laço sem progresso. Já em PHP, comparar certas strings que parecem números com == pode produzir igualdade onde o programador queria comparar o texto exato. Os três casos são diferentes, mas têm algo em comum: uma decisão local de sintaxe mudou uma propriedade importante do sistema.

Ao estudar cada exemplo, procurei separar três perguntas: qual saída o código apresenta, qual regra produz essa saída e que alteração deixaria a intenção mais explícita. Essa sequência é útil para depurar: corrigir um programa sem entender a causa pode apenas deslocar o erro para outro lugar.

O relatório segue essa mesma lógica. Primeiro analiso as expressões e as conversões, depois as decisões de controle e os laços. No fim, resolvo os seis trechos da atividade de fixação. Assim, cada resposta final pode ser relacionada a uma regra discutida antes, em vez de aparecer como um resultado decorado.

2. Quiz inicial: seis resultados que exigem atenção

O quiz de abertura funciona como um diagnóstico. Em vez de responder por intuição, é preciso aplicar a regra de cada linguagem. Reuni as seis situações abaixo porque elas antecipam quase todos os tópicos do restante da aula.

Python: potência e sinal

```
print(-2 ** 2, 2 ** 3 ** 2)   # -4 512
```

A potência é calculada antes do sinal negativo à esquerda: -(2 ** 2) resulta em -4. Duas potências sucessivas agrupam-se à direita: 2 ** (3 ** 2) resulta em 512. Acrescentar parênteses evita depender da memória de quem lê.

Java: soma seguida de concatenação

```
System.out.println(1 + 2 + "3" + 4 + 5);  // 3345
```

A avaliação segue da esquerda para a direita. Primeiro há uma soma numérica, 1 + 2 = 3. Ao encontrar a string "3", o operador passa a concatenar. O restante acrescenta "4" e "5" ao texto, formando "3345".

JavaScript: o mesmo símbolo, outra conversão

```
console.log("5" - 2, "5" + 2);   // 3 52
```

O operador de subtração força a conversão de "5" para número. O operador + também pode concatenar; quando encontra uma string, produz "52". A operação desejada precisa estar clara nos tipos dos operandos.

C e Python: comparações encadeadas

```
/* C */      3 > 2 > 1      // 0
# Python     3 > 2 > 1      # True
```

C agrupa como (3 > 2) > 1; o primeiro resultado é 1, e 1 > 1 é falso. Python entende duas comparações ligadas por “e”: 3 > 2 e 2 > 1. No mesmo exemplo, 0 or "padrão" em Python devolve a string, não um booleano.

JavaScript: igualdade com e sem conversão

```
0 == ""      // true
"" == "0"     // false
0 == "0"     // true
```

As comparações com == podem converter string e número antes do teste. As duas strings, porém, são comparadas como texto. Assim, a igualdade frouxa não se comporta como uma relação transitiva nesse conjunto. Quando a conversão não faz parte da intenção, === comunica melhor o teste.

3. Precedência, associatividade e divisão

Uma linguagem precisa decidir qual operação vem primeiro e como operadores da mesma prioridade se agrupam. Com a = 3, b = 4, c = 5, a expressão a + b * c normalmente vale 23, pois a multiplicação precede a soma. Se fosse avaliada como (a + b) * c, valeria 35. Não há erro de compilação em nenhuma das duas contas; só a intenção do programador permite saber qual é a correta.

Associatividade é outra regra. Em 10 - 3 - 2, o agrupamento usual à esquerda produz 5. Para a potência em Python, 2 ** 3 ** 2 agrupa à direita e produz 512. No JavaScript, escrever -2 ** 2 sem parênteses é erro de sintaxe. É preciso escolher entre -(2 ** 2) e (-2) ** 2. A exigência força uma decisão explícita justamente num caso propenso a confusão.

Divisão e resto com valores negativos

O exemplo -7 / 2 mostra que “divisão inteira” não significa a mesma coisa em todas as linguagens. Em Python, -7 // 2 vale -4, porque o quociente é arredondado para baixo. Em Java, a divisão inteira de -7 por 2 resulta em -3, por truncamento em direção a zero. Isso também afeta o resto: em Python -7 % 2 vale 1, enquanto em Java vale -1. Uma fórmula transportada de uma linguagem para outra pode, portanto, errar só para entradas negativas.

A notação prefixa de Lisp ilustra outra solução de projeto: em (+ a (* b c)), os parênteses e a posição do operador já descrevem a árvore da expressão. Não é preciso uma tabela tradicional de precedência. O contraste ajuda a perceber que precedência não é uma lei matemática universal, mas uma escolha da sintaxe.

Minha regra prática depois desses exemplos é usar parênteses quando uma expressão mistura operações com efeitos diferentes ou pode ser lida de mais de uma maneira. Parênteses não substituem conhecer a linguagem, mas reduzem o esforço de revisão e tornam um erro de intenção mais visível.

Três leituras da mesma expressão

```
a = 3; b = 4; c = 5
a + b * c       -> 23
(a + b) * c     -> 35
a + (b * c)     -> 23
```

Ao inserir os parênteses, a dúvida passa a ser sobre o cálculo pretendido, não sobre a tabela de precedência. Isso é especialmente útil quando a expressão combina aritmética com comparações ou operadores lógicos, casos em que uma leitura rápida costuma ignorar o agrupamento real.

4. Ordem dos operandos e efeitos colaterais

A ordem de avaliação passa a importar quando uma função altera um valor que a própria expressão utiliza. No exemplo da aula, a começa em 5; fun1() muda a para 17 e retorna 3. A expressão a = a + fun1() levanta a dúvida: qual valor de a é lido na soma?

```
int a = 5;
int fun1(void) { a = 17; return 3; }
/* depois: */ a = a + fun1();
```

Em Java, a avaliação dos operandos de uma operação binária ocorre da esquerda para a direita. Num exemplo equivalente, o valor antigo de a é obtido antes da chamada e a soma resulta em 8. Em C, não se deve transportar essa garantia automaticamente para o trecho; a ordem de avaliação relevante não é fixa como em Java, e o programa não deve depender de uma saída específica. Uma forma mais fácil de entender é separar a chamada, a leitura e a soma em instruções com ordem intencional.

Esse é o sentido prático de “efeito colateral” nesta aula: além de devolver um resultado, a função altera um estado que outras partes do programa observam. Funções sem esse tipo de alteração são mais simples de testar e de combinar, porque chamadas iguais podem ser analisadas pelo valor que retornam. Isso não significa que todo efeito colateral seja errado; entrada, saída e atualização de dados são necessárias. Significa que misturá-los numa expressão curta pode esconder dependências.

Sobrecarga de operadores

O símbolo + já tem vários usos: soma números e concatena strings em Java. O resultado de 'a' + 1 é 98, porque o caractere participa de uma soma numérica; "" + 'a' + 1 produz "a1", pois a string inicial leva à concatenação. Python permite que uma classe defina, por exemplo, __add__ para somar vetores. Isso pode tornar v1 + v2 natural, mas o operador deve manter uma interpretação previsível. Quando a operação tem custo ou comportamento inesperado, um método com nome explícito pode ser mais legível.

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, outro):
        return Vetor(self.x + outro.x, self.y + outro.y)
```

Nesse exemplo, Vetor(1, 2) + Vetor(3, 4) cria um vetor com coordenadas (4, 6). O ganho de legibilidade aparece quando “somar vetores” tem sentido claro para quem usa a classe. A mesma técnica seria ruim se o operador +, sem indicação no código, alterasse um arquivo ou fizesse uma chamada de rede.

5. Conversões de tipos e perda de informação

A conversão entre tipos pode ser explícita, como um cast, ou implícita, quando a linguagem a faz durante a expressão. Alargamento costuma levar um valor para um tipo de maior faixa; estreitamento pode descartar informação. A palavra “alargamento”, porém, não garante preservação exata: converter um inteiro grande para float em Java pode perder precisão, pois o tipo de ponto flutuante não representa todos os inteiros da faixa de int.

```
int grande = 123456789;
float f = grande;
System.out.println((int) f);  // 123456792
```

O valor obtido na volta não é o mesmo que entrou. Outros exemplos mostram mecanismos diferentes: (int) 3.99 descarta a parte fracionária e vira 3; (byte) 200 produz -56 ao conservar os oito bits de menor peso. Nesses casos, o cast indica que o programador pediu a conversão, mas não prova que ela seja adequada ao problema.

O tipo do destino não muda a operação anterior

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;  // 0.0
```

Os dois operandos da divisão são inteiros. Primeiro ocorre 7 / 10 = 0; depois a multiplicação continua a partir desse zero. Só ao final o resultado é colocado em double. Para obter 70.0, pelo menos um operando da divisão precisa ser convertido antes: (double) acertos / total * 100. Esse é um bug discreto porque o tipo da variável parece correto à primeira vista.

Em Java, a expressão b + c com duas variáveis byte é promovida para int; por isso byte s = b + c não compila sem conversão explícita. Já o operador composto i += 3.7 admite uma conversão implícita de volta para o tipo de i. Com i = 3, o resultado se torna 6 após o truncamento. A forma curta economiza caracteres, mas pode esconder a perda de informação.

A comparação entre linguagens mostrou duas filosofias. Go e Rust costumam exigir conversões escritas pelo programador entre tipos numéricos distintos. Java oferece mais promoções automáticas. A escolha de projeto troca conveniência por visibilidade: quanto menos conversões são automáticas, mais fácil é localizar no código o ponto em que a precisão pode mudar.

Conferindo as etapas da conta

Para revisar uma expressão mista, ajuda anotar o tipo em cada etapa. Em acertos / total * 100, os três valores iniciais são inteiros, logo o resultado intermediário da divisão também é inteiro. Em (double) acertos / total * 100, o primeiro operando já é real quando a divisão começa. A variável que recebe a resposta só participa depois que a expressão do lado direito foi calculada.

6. Coerção em JavaScript e PHP

Em JavaScript, operadores diferentes solicitam conversões diferentes. "5" - 2 vale 3, porque a subtração trabalha numericamente; "5" + 2 vale "52", porque + também concatena. Valores booleanos e nulos entram na mesma discussão: true + 1 vale 2, null + 1 vale 1 e undefined + 1 produz NaN. São resultados definidos pela linguagem, mas fáceis de interpretar errado durante uma revisão rápida.

```
console.log(0 == "", "" == "0", 0 == "0");
// true false true
```

O caso acima é útil porque impede uma simplificação perigosa: não basta dizer que == “ignora o tipo”. A operação segue regras específicas de coerção conforme o par de operandos. A comparação === preserva a distinção entre número e string, de modo que 0 === "0" é falso. Ela deve ser a escolha natural quando não há motivo explícito para converter os valores.

Strings numéricas em PHP

O manual do PHP distingue ==, que compara após as conversões aplicáveis, de ===, que também exige identidade de tipo. Duas strings numéricas como "0e123" e "0e456" podem ser lidas como notação científica de zero numa comparação frouxa. Como textos, porém, são diferentes. O exemplo é relevante para qualquer comparação em que os caracteres exatos importam, como identificadores ou hashes.

A conclusão não é que a linguagem esteja “quebrada”: ela executa as regras que definiu. O problema aparece quando o código não mostra a intenção de comparar números ou textos. Antes de usar ==, é preciso saber quais tipos podem chegar àquela linha. Em PHP e JavaScript, uma comparação estrita evita que a igualdade surja por uma conversão não planejada.

Também importa a versão da linguagem. As regras de comparação do PHP passaram por alterações, especialmente para strings não numéricas. Por isso, ao discutir um resultado específico, vale consultar o manual da versão usada, em vez de repetir uma regra lembrada de outro projeto.

Uma verificação simples antes de comparar

Se um valor veio de formulário, arquivo ou API, pode chegar como texto mesmo quando representa um número. Converter explicitamente para o tipo esperado e validar o formato na entrada reduz a dependência de coerções posteriores. Se o valor é um identificador, a comparação deve respeitar os caracteres e não reinterpretá-lo como número. Essa distinção explica por que o exemplo dos hashes é mais grave do que uma curiosidade de sintaxe.

7. Expressões booleanas e curto-circuito

Cada linguagem define o que aceita como condição. Em C, zero é falso e um valor diferente de zero é verdadeiro; por isso if (-1) entra no bloco. Java, Go e Rust exigem uma expressão booleana. Python e JavaScript aceitam valores convertidos para verdadeiro ou falso segundo regras próprias. Essa flexibilidade é útil, mas pede cuidado com valores como zero, string vazia e coleções vazias.

```
/* C */     3 > 2 > 1        /* falso */
# Python    3 > 2 > 1        # True
```

O exemplo reaparece aqui porque mistura comparação e interpretação booleana. Em C, 3 > 2 resulta em 1, e a comparação seguinte é 1 > 1, falsa. Para expressar duas condições em C, seria necessário (3 > 2) && (2 > 1). Em Python, o encadeamento já representa as duas relações.

Quando a segunda expressão nem chega a ser avaliada

```
s != null && s.length() > 0
s != null &  s.length() > 0
```

Com s nulo, a primeira expressão Java termina em falso sem chamar length(). O operador & com booleanos avalia os dois lados e, nesse caso, provoca exceção. O curto-circuito também protege o acesso a um array: em i < lista.length && lista[i] != chave, o segundo teste não é feito quando i já atingiu o tamanho da lista.

Esse comportamento não serve apenas para evitar erros. Ele pode pular efeitos colaterais: se o segundo operando incrementa uma variável, esse incremento talvez não aconteça. Por isso, alterações de estado escondidas dentro de condições são difíceis de acompanhar.

JavaScript oferece ainda ?. para interromper o acesso quando uma parte da cadeia é nula ou indefinida e ?? para fornecer um padrão apenas nesses dois casos. Com qtd = 0, qtd || 10 devolve 10, mas qtd ?? 10 preserva 0. Escolher entre eles depende de saber se zero é uma ausência de valor ou um valor válido.

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade;  // undefined, sem exceção
const qtd = 0;
qtd || 10;          // 10
qtd ?? 10;          // 0
```

8. Atribuição e seleção de dois caminhos

Em C, a atribuição é uma expressão e devolve o valor atribuído. Isso permite escrever sum = count = 0 e usar uma leitura dentro da condição de um laço, como while ((ch = getchar()) != EOF). O mesmo recurso abre espaço para o engano if (x = y): o código atribui y a x e testa o valor resultante, embora quem escreveu talvez pretendesse comparar com ==. Chaves e avisos do compilador ajudam, mas uma revisão deve perguntar se a atribuição é intencional.

Python separa a atribuição comum de expressões, mas possui o operador := para atribuir durante um teste de forma explícita. A troca a, b = b, a também é uma característica própria: os valores do lado direito são obtidos antes de atualizar os nomes do lado esquerdo. Em Rust, uma atribuição comum retorna o valor unitário (), não o valor escrito; por isso não se usa diretamente como condição booleana.

O else pendente

```
if (sum == 0)
    if (count == 0)
        result = 0;
    else
        result = 1;
```

A indentação pode sugerir que o else pertence ao primeiro if. Em C, Java e linguagens de sintaxe semelhante, ele se associa ao if mais próximo que ainda não tem else. Com sum = 0 e count = 5, o bloco interno cai no else e define result = 1. Se a intenção era outra, chaves devem delimitar os blocos. A organização visual ajuda o leitor, mas não altera a gramática.

Em Rust, if pode produzir um valor, como em let y = if x > 0 { x } else { 2 * x };. Python oferece uma expressão condicional equivalente para casos simples. Esses formatos são úteis quando os dois ramos são curtos; para decisões maiores, um bloco comum costuma ser mais fácil de seguir.

O contraste entre essas linguagens mostra que a atribuição não deve ser tratada como um gesto mecânico. Ela pode ter valor próprio, pode ser apenas uma instrução e pode introduzir um nome em vez de modificar um objeto existente. Ao encontrar uma condição que contém = ou :=, vale verificar duas coisas: se a linguagem permite essa forma e se o valor testado é realmente o que se queria testar.

9. Seleção múltipla: switch e match

No switch tradicional de C, o fluxo pode continuar nos casos seguintes quando não há break. Um caso sem instrução própria também pode ser usado para agrupar alternativas, mas a mesma regra causa erros quando o programador esquece de interromper o fluxo. No exemplo da atividade, entrar no caso 2 e continuar até default faz a última atribuição sobrescrever as anteriores.

```
switch (dia) {
  case 1: nome = "domingo";
  case 2: nome = "segunda";
  case 3: nome = "terça";
  default: nome = "inválido";
}
```

Com dia = 2, a variável termina como "inválido". O switch moderno com setas em Java não cai automaticamente no caso seguinte e pode ser usado como expressão. Rust oferece match, que exige cobrir todos os padrões possíveis; quando faltam casos, o compilador aponta a incompletude. São escolhas de sintaxe que tornam uma classe de enganos mais difícil de escrever.

Laços controlados por contador e por iterável

Em C, a forma do for contém inicialização, condição e atualização, mas nada impede modificar a variável de controle dentro do corpo. Num laço que imprime i e também executa i += 2 antes do i++ do cabeçalho, os valores impressos podem ser 0, 3, 6 e 9. O corpo mudou o passo esperado por quem olhou apenas o cabeçalho.

O for de Python percorre os valores de um iterável. Em for i in range(10), fazer i += 2 no corpo não altera a próxima entrega de range; a iteração continua com 0, 1, 2 e assim por diante. Essa diferença é importante ao traduzir algoritmos entre linguagens. A pergunta correta não é só “qual a condição de parada?”, mas também “de onde vem o próximo valor?”.

```
/* C: altera o passo dentro do corpo */
for (int i = 0; i < 10; i++) {
    printf("%d ", i);
    i += 2;
}
/* saída: 0 3 6 9 */
```

10. Laços, desvios e progresso

JavaScript tem duas construções parecidas na escrita e diferentes no resultado. for...in percorre chaves de propriedades; num array comum, elas aparecem como índices em texto. for...of percorre os valores. Somar com for...in pode transformar uma conta em concatenação. No exemplo [10, 20, 30], começar com zero e acrescentar as chaves "0", "1" e "2" produz "0012"; percorrer os valores com for...of produz 60.

O laço que não avança

O caso dos tocadores Zune ilustra uma condição de saída mal pensada. Para certo dia de um ano bissexto, a condição externa do while permanecia verdadeira, mas o ramo interno não alterava nem o número de dias nem o ano. Assim, a próxima volta encontrava exatamente o mesmo estado. Ao revisar um laço, é preciso identificar qual variável mede o progresso e demonstrar que todo caminho que repete aproxima o programa da saída.

```
while (dias > 365) {
    if (bissexto(ano)) {
        if (dias > 366) { dias -= 366; ano++; }
    } else {
        dias -= 365; ano++;
    }
}
```

Se dias é 366 num ano bissexto, o laço externo entra, mas o teste dias > 366 é falso. Nenhuma variável muda, e o mesmo caminho se repete. O defeito aparece exatamente no limite, razão pela qual testes com casos de fronteira são tão valiosos.

Goto e alternativas restritas

O goto permite um salto incondicional. No exemplo “goto fail”, uma linha duplicada ficou fora da condição, embora a indentação sugerisse o contrário; a verificação seguinte não era alcançada. A lição específica é verificar blocos e alcance dos desvios. Comandos como break, continue e return também desviam o fluxo, mas têm destinos mais restritos e costumam ser mais fáceis de acompanhar.

Comandos protegidos e select

Os comandos protegidos apresentados na aula associam ações a guardas. Se mais de uma guarda é verdadeira, pode haver escolha entre alternativas prontas. O select de Go oferece uma ideia relacionada para operações em canais: quando várias comunicações podem prosseguir, uma delas é escolhida por seleção pseudoaleatória uniforme. Isso evita tratar o primeiro caso escrito como prioridade fixa. A ligação é conceitual; o comportamento concreto de cada construção deve ser lido na especificação de sua linguagem.

11. Atividade final: localizar o bug (1 a 3)

A atividade final pediu mais do que descobrir a saída: para cada trecho, era preciso encontrar a regra que explica o resultado e propor uma correção. Os três primeiros casos misturam iteração, valor padrão e seleção múltipla.

Exercício 1 — JavaScript: somar preços

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos) total += p;
console.log("Total: " + total);
```

Saída: “Total: 0012”. O laço percorre as chaves do array, não os preços. Como as chaves são strings, o operador += passa a concatenar. Correção: usar for (const p of precos); assim, a soma dos valores resulta em “Total: 60”.

Exercício 2 — Python: desconto igual a zero

```
def desconto(preco, d=None):
    d = d or 10
    return preco * (100 - d) / 100
print(desconto(200, 0))
```

Saída: 180.0. O argumento 0 representa “sem desconto”, mas é falso na condição de or; por isso é trocado por 10. Correção: testar explicitamente if d is None: d = 10. Agora zero permanece zero e a chamada devolve 200.0.

Exercício 3 — Java: switch sem interrupção

```
int dia = 2; String nome = "";
switch (dia) {
  case 1: nome = "domingo";
  case 2: nome = "segunda";
  case 3: nome = "terça";
  default: nome = "inválido";
}
System.out.println(nome);
```

Saída: “inválido”. A execução entra no caso 2, passa pelo caso 3 e chega a default, pois não há break. Correção: interromper cada caso ou usar a sintaxe de setas de um switch moderno, conforme a versão de Java disponível.

12. Atividade final: localizar o bug (4 a 6)

Os três casos seguintes mostram por que uma linha aparentemente pequena pode alterar o alcance de uma condição, criar uma variável inesperada ou mudar o tipo de uma conta.

Exercício 4 — C: ponto e vírgula após if

```
int saldo = 100, saque = 50;
if (saque > saldo);
printf("saldo insuficiente\n");
printf("fim\n");
```

Saída: as duas mensagens são impressas. O ponto e vírgula encerra um comando vazio que pertence ao if. Os dois printf ficam fora da condição. Correção: retirar o ponto e vírgula e usar chaves para delimitar o bloco que só deve executar quando o saldo for insuficiente.

Exercício 5 — Go: variável escondida no bloco

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

Saída: “dentro: 20” e “fora: 10”. O := dentro do bloco declara outro x, que esconde temporariamente a variável externa. Ao sair do bloco, reaparece o valor antigo. Correção: usar x = x * 2 se a intenção é alterar a variável já existente; nesse caso, a segunda linha também mostra 20.

Exercício 6 — Java: divisão inteira antes da conversão

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Saída: “0.0%”. A divisão de dois inteiros produz 0 antes da multiplicação e antes da atribuição a double. Correção: escrever (double) acertos / total * 100. A conversão acontece antes da divisão e a taxa passa a 70.0%.

Os seis casos têm a mesma estratégia de resolução: acompanhar os tipos e o escopo em cada etapa, em vez de inferir o resultado pela aparência geral do trecho. Isso ajuda a diferenciar um erro de sintaxe, que o compilador aponta, de um erro de lógica que produz uma saída plausível.

13. Síntese do aprendizado

A ideia que mais une a aula é a distância entre a intenção e a interpretação formal. O código pode parecer óbvio para uma pessoa e, ainda assim, usar outra regra de precedência, realizar uma conversão escondida ou associar um else ao bloco errado. Para analisar esses casos, passei a olhar primeiro para os tipos dos valores, depois para a ordem das operações e, por fim, para o alcance de cada bloco de controle.

Algumas medidas se repetem nos exemplos: colocar parênteses onde o agrupamento não é evidente; converter o operando antes da conta quando a divisão deve ser real; usar comparação estrita quando texto e número não devem ser misturados; escrever chaves para tornar o alcance de if e laços explícito; evitar efeitos colaterais escondidos em expressões; e verificar se todo caminho de um laço faz progresso. Nenhuma dessas medidas depende de uma ferramenta específica. Elas tornam o raciocínio do programa mais visível para quem escreve e para quem revisa.

As linguagens também ajudam de modos diferentes. O match exaustivo de Rust torna casos esquecidos um erro de compilação; o switch com setas reduz a queda acidental entre casos; o for...of expressa claramente a intenção de percorrer valores; operadores como ?? distinguem ausência de um valor válido igual a zero. Mesmo assim, conhecer esses recursos não substitui testar entradas que explorem fronteiras: zero, nulo, ano bissexto, números negativos e tipos misturados.

Referências usadas para conferir as regras

SEBESTA, Robert W. Conceitos de Linguagens de Programação, 11ª ed., capítulos 7 e 8; material da Aula 07 — Expressões, Atribuição e Controle.

Python Software Foundation. Python Language Reference: expressões e comparações. https://docs.python.org/3/reference/expressions.html

Oracle. Java Language Specification, seção 15: expressões e ordem de avaliação. https://docs.oracle.com/javase/specs/jls/se26/html/jls-15.html

Ecma International. ECMAScript Language Specification. https://tc39.es/ecma262/

PHP Documentation. Comparison Operators. https://www.php.net/manual/en/language.operators.comparison.php

The Go Authors. The Go Programming Language Specification. https://go.dev/ref/spec

The Rust Project. The Rust Programming Language, seção sobre match. https://doc.rust-lang.org/book/ch06-02-match.html

ISO/IEC 9899:2024. Programming languages — C. https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf